

FastAPI Cheat Sheet

Structure, patterns & production practices for building APIs the right way

1. Project Structure

2. Core App & Routing

3. Pydantic Models (Schemas)

4. Path, Query & Body Params

5. Dependency Injection

6. Database (SQLAlchemy async)

7. Authentication (OAuth2 + JWT)

8. Error Handling

9. Background Tasks & Async

10. Middleware & CORS

11. Testing

12. Performance & Deployment

13. Status Codes Reference

14. Golden Rules Checklist

1 Project Structure

Never dump everything into one `main.py`. Split by domain/feature so the app scales cleanly.

```
app/
├── main.py           # app creation, router includes
├── core/
│   ├── config.py     # Settings (pydantic-settings)
│   └── security.py    # hashing, JWT helpers
├── api/
│   └── v1/
│       ├── routers/
│       │   ├── users.py
│       │   └── items.py
│       └── deps.py    # shared dependencies
├── models/           # SQLAlchemy ORM models
├── schemas/          # Pydantic request/response models
├── crud/             # DB query functions
├── db/
│   ├── session.py
│   └── base.py
└── tests/
```

BEST PRACTICE

Keep *schemas* (Pydantic) separate from *models* (ORM). Never expose your DB model directly in a response — always map through a schema.

2 Core App & Routing

App instance

```
from fastapi import FastAPI

app = FastAPI(
    title="My API",
    version="1.0.0",
    description="...",
    docs_url="/docs",
    redoc_url="/redoc",
)
```

APIRouter (modular routes)

```
from fastapi import APIRouter

router = APIRouter(
    prefix="/users",
    tags=["users"],
)

@router.get("/{user_id}")
async def get_user(user_id: int):
    return {"id": user_id}

# in main.py
app.include_router(router, prefix="/api/v1")
```

HTTP method decorators

VERB	USE FOR
GET	Read / fetch data, no side effects
POST	Create a new resource
PUT	Replace a resource fully
PATCH	Partial update
DELETE	Remove a resource

Route config extras

```
@router.post(
    "/",
    response_model=UserOut,
    status_code=201,
    summary="Create a user",
    response_model_exclude_none=True,
)
async def create_user(payload: UserCreate):
    ...
```

TIP

Always set `response_model` — it filters output, generates accurate docs, and hides internal fields.

3 Pydantic Models (Schemas)

Use separate schemas per operation: `Create`, `Update`, `Out` (a.k.a. `Read`). This is the single biggest habit that separates clean FastAPI code from messy code.

```
from pydantic import BaseModel, EmailStr, Field
from datetime import datetime

class UserBase(BaseModel):
    email: EmailStr
    full_name: str = Field(min_length=1, max_length=80)

class UserCreate(UserBase):
    password: str = Field(min_length=8)

class UserUpdate(BaseModel):
    full_name: str | None = None

class UserOut(UserBase):
    id: int
    created_at: datetime

class Config:
    from_attributes = True  # enables ORM -> schema mapping
```

4 Path, Query & Body Parameters

```
# Path param
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    ...

# Query params with validation
@app.get("/items/")
async def list_items(
    q: str | None = None,
    limit: int = Query(10, ge=1, le=100),
    offset: int = 0,
):
    ...
```

```
# Body (Pydantic model = auto JSON parsing)
@app.post("/items/")
async def create_item(item: ItemCreate):
    ...

# Multiple sources combined
@app.put("/items/{item_id}")
async def update_item(
    item_id: int,
    item: ItemUpdate,
    x_token: str = Header(...),
):
    ...
```

BEST PRACTICE

Type hints ARE the validation layer. FastAPI reads them to validate, coerce types, and build OpenAPI docs automatically — always type every parameter.

5 Dependency Injection

Use `Depends()` for anything reused across routes: DB sessions, current user, pagination, permission checks.

```
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession

async def get_db():
    async with SessionLocal() as session:
        yield session        # yield = cleanup runs after request

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db),
):
    user = await verify_token_and_fetch_user(token, db)
    if not user:
        raise HTTPException(401, "Invalid credentials")
    return user

@app.get("/me")
async def read_me(user: User = Depends(get_current_user)):
    return user
```

TIP

Dependencies can depend on other dependencies (chaining). FastAPI resolves and caches them once per request automatically.

6 Database — Async SQLAlchemy

```
# db/session.py
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker

engine = create_async_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

# crud/user.py
async def get_user(db: AsyncSession, user_id: int):
    result = await db.execute(select(User).where(User.id == user_id))
    return result.scalar_one_or_none()

async def create_user(db: AsyncSession, data: UserCreate):
    user = User(**data.model_dump())
    db.add(user)
    await db.commit()
    await db.refresh(user)
    return user
```

WATCH OUT

Use `async` DB drivers (`asyncpg`, `aiomysql`) with `async SQLAlchemy` — mixing `sync` drivers with `async` routes blocks the event loop and kills concurrency.

7 Authentication — OAuth2 + JWT

```
from fastapi.security import OAuth2PasswordBearer
from jose import jwt
from datetime import datetime, timedelta

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

def create_access_token(sub: str, expires_min: int = 30):
    payload = {
        "sub": sub,
        "exp": datetime.utcnow() + timedelta(minutes=expires_min),
    }
    return jwt.encode(payload, SECRET_KEY, algorithm="HS256")

@app.post("/auth/login")
async def login(form: OAuth2PasswordRequestForm = Depends()):
    user = await authenticate(form.username, form.password)
    if not user:
        raise HTTPException(401, "Incorrect username or password")
    token = create_access_token(sub=user.email)
    return {"access_token": token, "token_type": "bearer"}
```

BEST PRACTICE

Hash passwords with `passlib[bcrypt]`, never store plaintext. Keep `SECRET_KEY` in environment variables, never in code.

8 Error Handling

```
# Raise built-in HTTP errors
raise HTTPException(
    status_code=404,
    detail="Item not found",
)

# Custom exception class
class OutOfStockError(Exception):
    def __init__(self, item: str):
        self.item = item
```

```
# Global handler for it
@app.exception_handler(OutOfStockError)
async def handler(request, exc):
    return JSONResponse(
        status_code=409,
        content={"detail": f"{exc.item} is out of stock"},
    )
```

TIP

Return consistent error shapes across the whole API, e.g. `{"detail": "...", "code": "..."}` , so clients can handle errors predictably.

9 Background Tasks & Async Rules

```
from fastapi import BackgroundTasks

@app.post("/notify")
async def notify(email: str, tasks: BackgroundTasks):
    tasks.add_task(send_email, email)  # runs after response is sent
    return {"message": "queued"}
```

SITUATION	RULE
Route does I/O (DB, HTTP call, file)	Use <code>async def</code> + <code>await</code> an async library
Route does CPU-heavy / blocking sync work	Use plain <code>def</code> — FastAPI runs it in a threadpool
Fire-and-forget short task (email, log)	<code>BackgroundTasks</code>
Heavy/long job (video processing, reports)	Real task queue — Celery / Arq / RQ, not <code>BackgroundTasks</code>

WATCH OUT

Never call a blocking sync function (e.g. `requests.get`, sync DB driver) directly inside `async def` — it freezes the entire event loop for every user.

10 Middleware & CORS

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://myfrontend.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.middleware("http")
async def add_process_time(request, call_next):
    start = time.time()
    response = await call_next(request)
    response.headers["X-Process-Time"] = str(time.time() - start)
    return response
```

11 Testing

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create_user():
    response = client.post("/users/", json={
        "email": "a@b.com", "password": "secret123"
    })
    assert response.status_code == 201
    assert response.json()["email"] == "a@b.com"
```

TIP

Override dependencies in tests with `app.dependency_overrides[get_db] = get_test_db` to use a test database instead of production.

12 Performance & Deployment

- ✓ Run with **Uvicorn** behind **Gunicorn** using multiple workers: `gunicorn -k uvicorn.workers.UvicornWorker -w 4 app.main:app`
- ✓ Use `response_model_exclude_unset` / pagination for large payloads
- ✓ Add `orjson` as the default response class for faster JSON serialization
- ✓ Cache expensive reads (Redis) instead of recomputing per request
- ✓ Set request timeouts and connection pool limits on outbound HTTP clients
- ✓ Enable Gzip: `app.add_middleware(GZipMiddleware, minimum_size=1000)`
- ✓ Always pin dependency versions; use `pydantic-settings` for env-based config, never hardcode secrets

13 Status Codes Quick Reference

CODE	MEANING	WHEN TO USE
200	OK	Successful GET / general success
201	Created	Successful POST that creates a resource
204	No Content	Successful DELETE, nothing to return
400	Bad Request	Malformed input, failed custom validation
401	Unauthorized	Missing / invalid credentials
403	Forbidden	Authenticated, but not allowed
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate resource, state conflict
422	Unprocessable Entity	Pydantic validation failure (automatic)
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unhandled exception

14 Golden Rules Checklist

- ✓ Separate **schemas** (Pydantic) from **models** (ORM) — never return DB models directly
- ✓ Every route has a `response_model` and correct `status_code`
- ✓ Every parameter is type-hinted — let FastAPI validate for you
- ✓ Shared logic (DB session, auth, pagination) lives in `Depends()`, not copy-pasted
- ✓ `async def` only for non-blocking I/O; plain `def` for blocking/sync work
- ✓ Secrets and config come from environment variables via `pydantic-settings`
- ✓ Consistent, structured error responses across the whole API
- ✓ Routers are split by domain and mounted with `include_router`
- ✓ Tests cover routes using `TestClient` with dependency overrides
- ✓ OpenAPI docs (`/docs`) are treated as documentation — write clear summaries and descriptions

FastAPI Cheat Sheet — build APIs like a genius, one clean endpoint at a time.